

Prompting for Gen-AI

Basics of Prompt Engineering for LLMs

General Agenda

Part 1: Introduction to Gen-AI

- Welcome and Overview
- What are Gen-AI Tools or LLMs?
- Importance of LLM Settings

Part 2: Techniques in Prompt Engineering

- Basics and elements of Prompting.
- Key Components of Prompt Engineering
- General Tips for Designing Prompts
- Strategies for Effective Prompting
- Techniques in Advanced Prompt Formulation

Part 3: Exploring ChatGPT with Data Analytics

- Brief Introduction to ChatGPT Plugins for Data Analytics
- Other useful plugins or tools

Closing

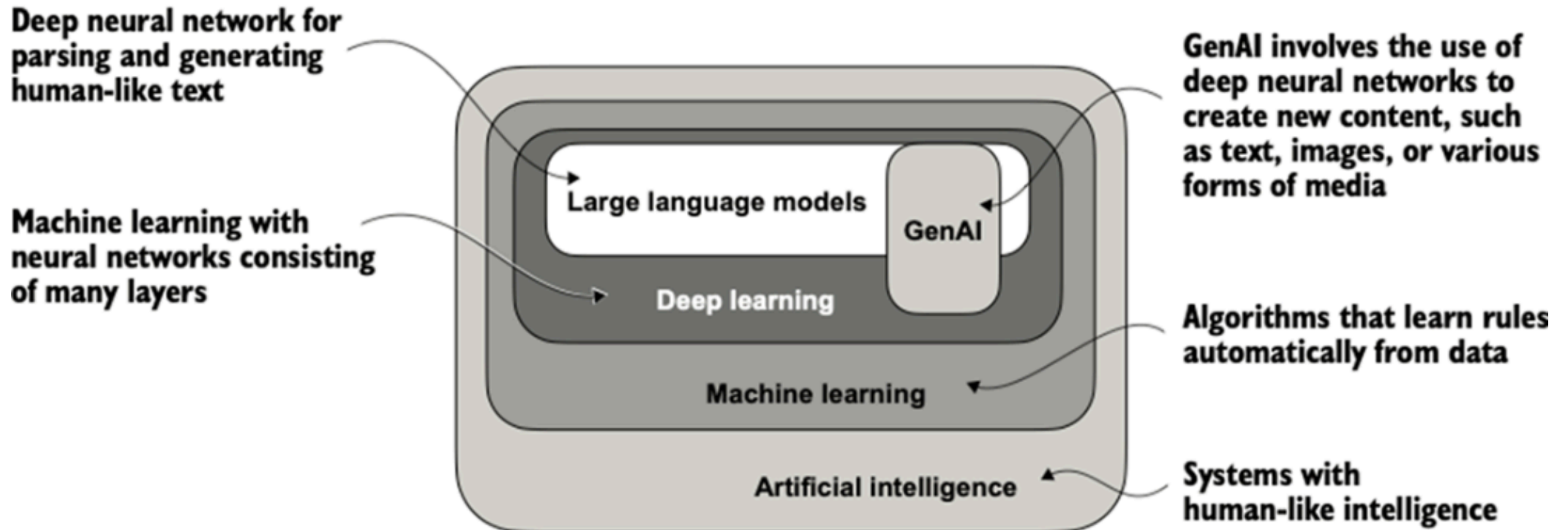
- Feedback and Closing Remarks

A Little about me

I am University Teacher in Statistics (Teaching Fellow - FHEA) at the School of Mathematics (SoM), at the University of Edinburgh since 2022.

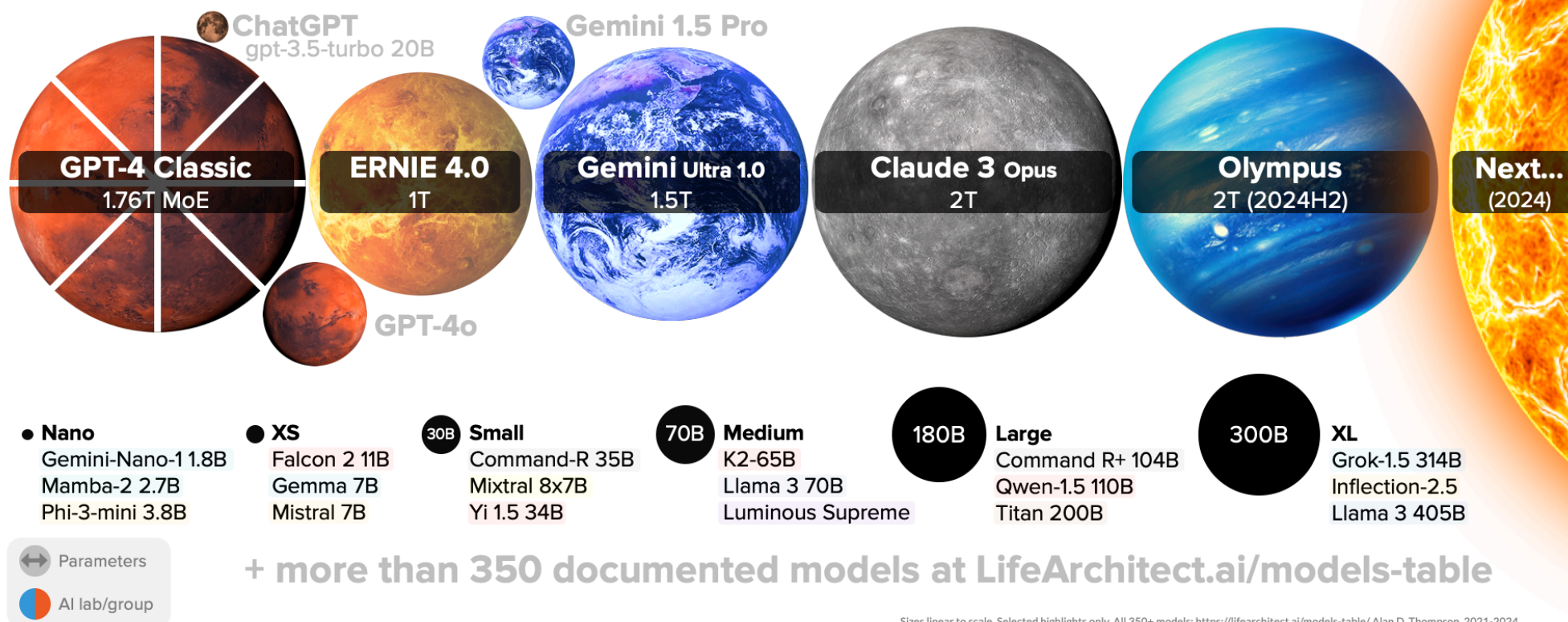
- Beyond being a member teaching team in the SoM
 - TEMSE seminars co-organiser and Leading Gen-AI related discussions
 - Being a part of EdinbR and RSS Edinburgh local group
 - Contributing for Pair Programming club & CDCS

Intersection of fields



Huge Public Interest

LARGE LANGUAGE MODEL HIGHLIGHTS (JUN/2024)



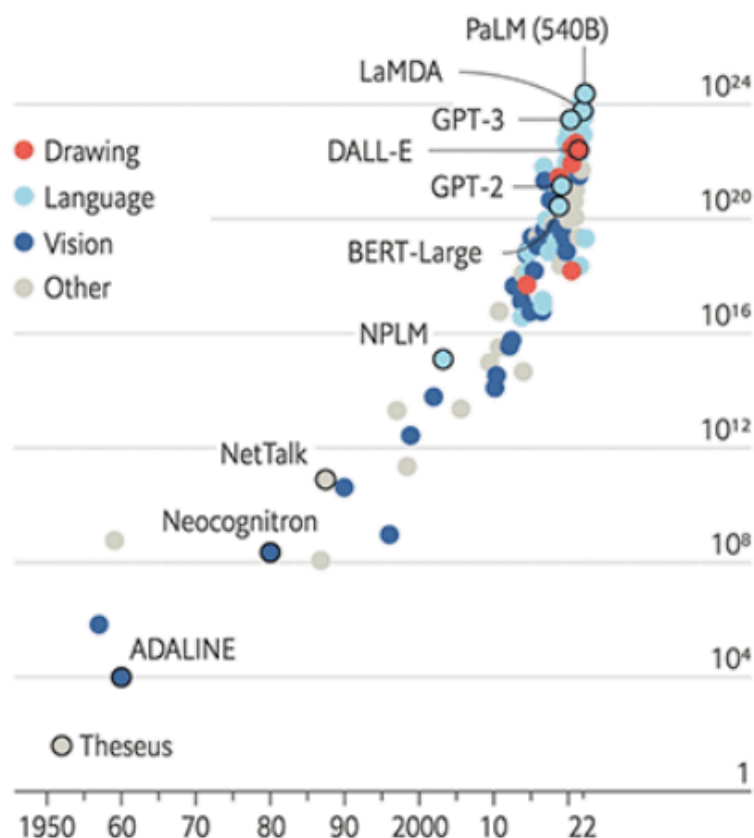
Sizes linear to scale. Selected highlights only. All 350+ models: <https://life architect.ai/models-table/> Alan D. Thompson. 2021-2024.



[LifeArchitect.ai/models](https://life architect.ai/models)

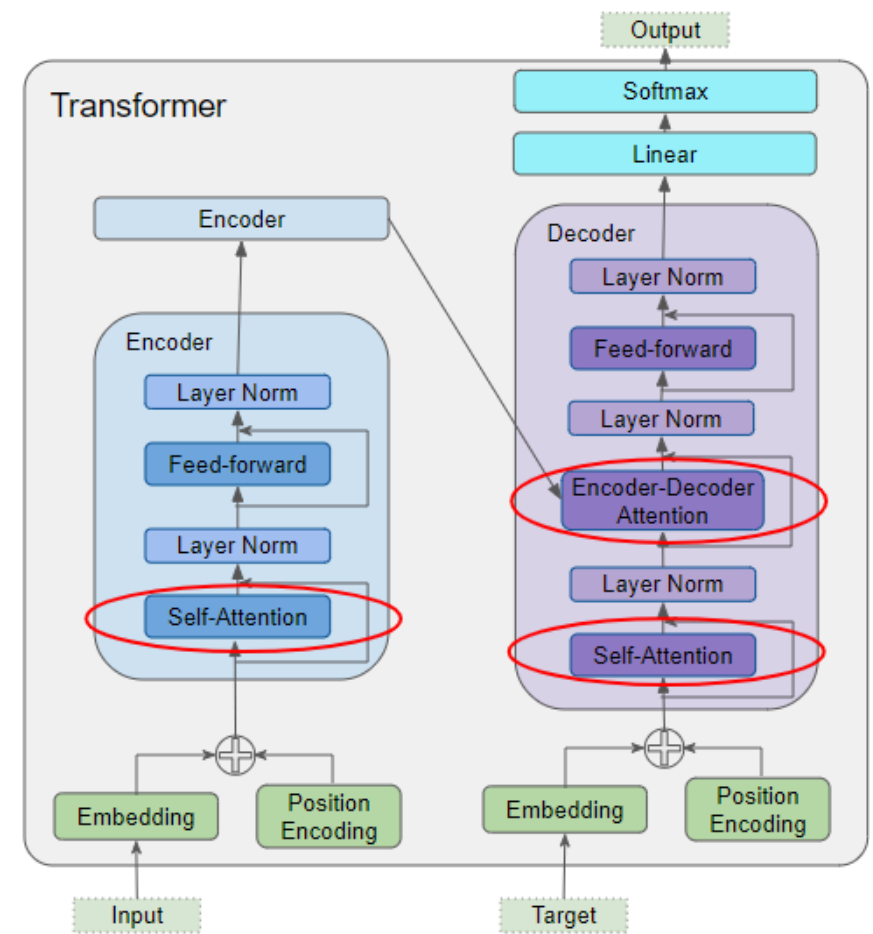
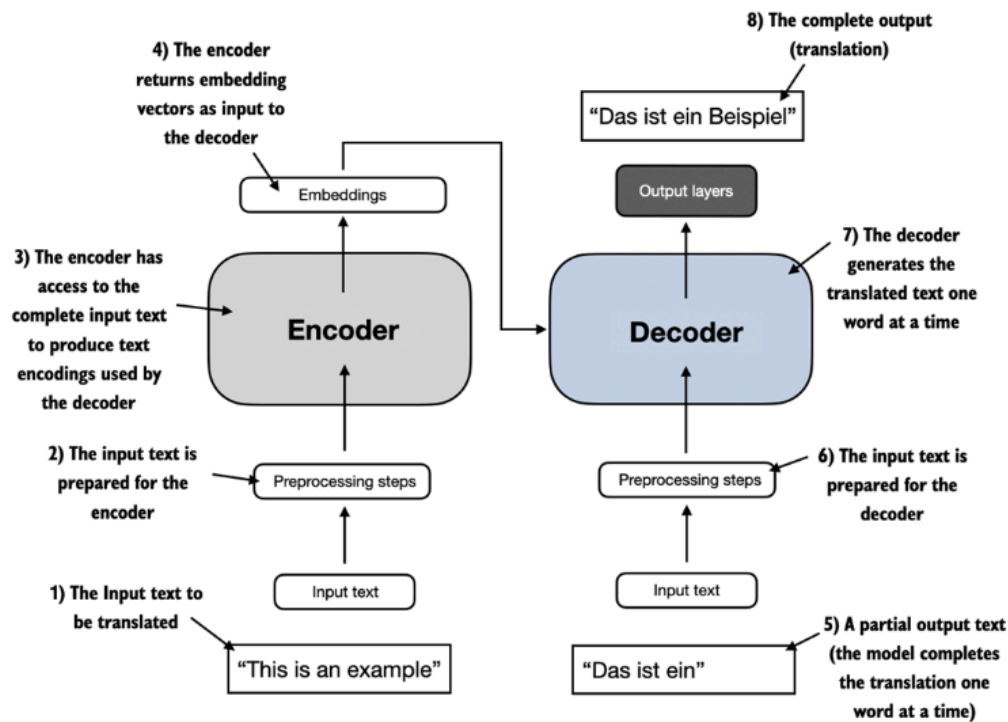
Power behind it ?

AI training runs, estimated computing resources used
Floating-point operations, selected systems, by type, log scale



- Parallelization and specialized hardware
 - Graphics Processing Units (GPUs)
 - Tensor Processing Units (TPUs)
- Cloud computing
- Hardware-software optimization

Main advancement since 2017 ?



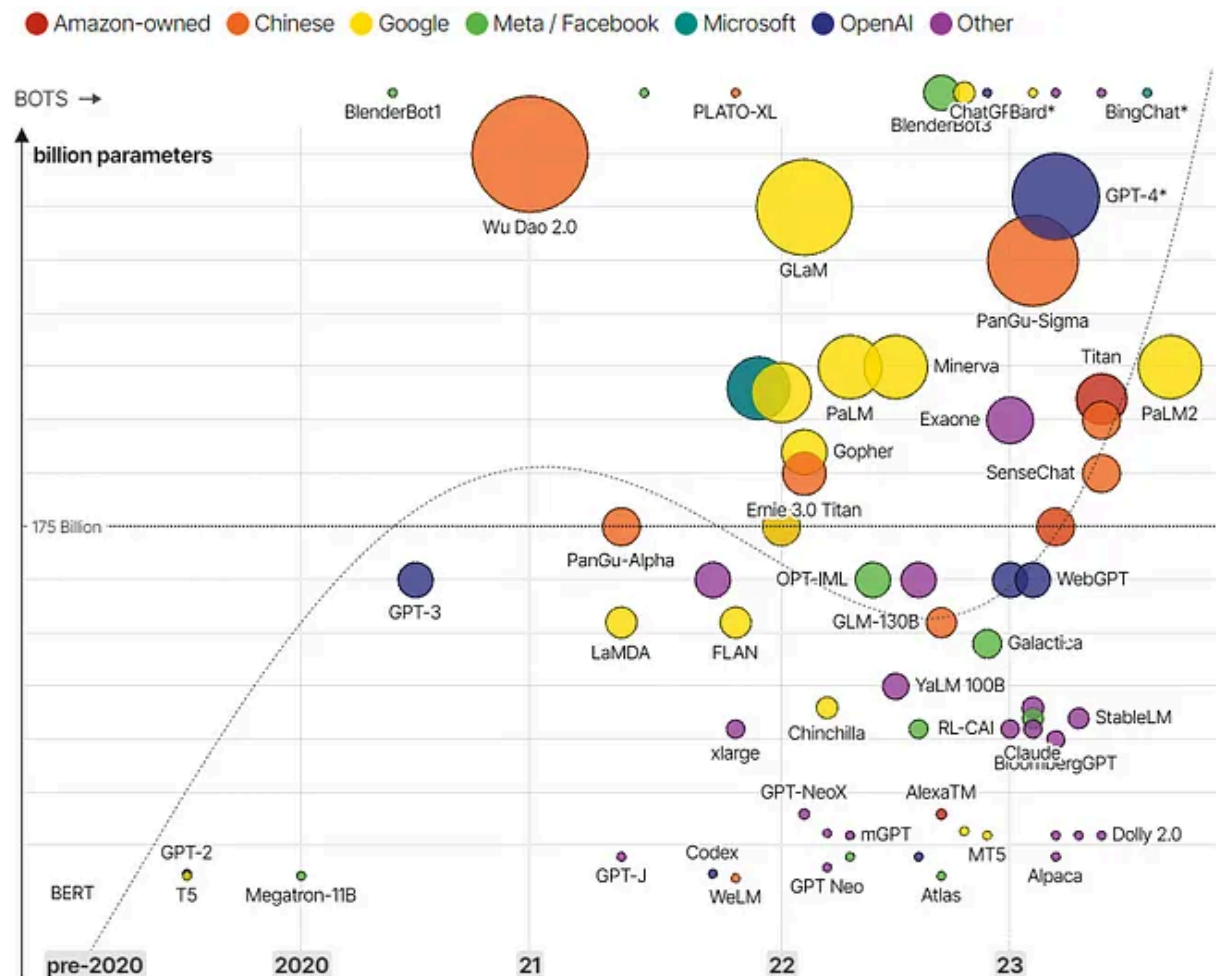
Architecture

Advantages and Limitations of Transformers

- Parallelization and Efficiency
- Long-Range Dependencies
- Transfer Learning and Contextual Embeddings
- Attention Overhead for Long Sequences
- Lack of Sequential Order
- Excessive Parameterization
- Fixed Input Length and Inability to Handle Unstructured Inputs

Importance of Attention ?

LLM tools landscape



David McCandless, Tom Evans, Paul Barton
Information is Beautiful // May 2023

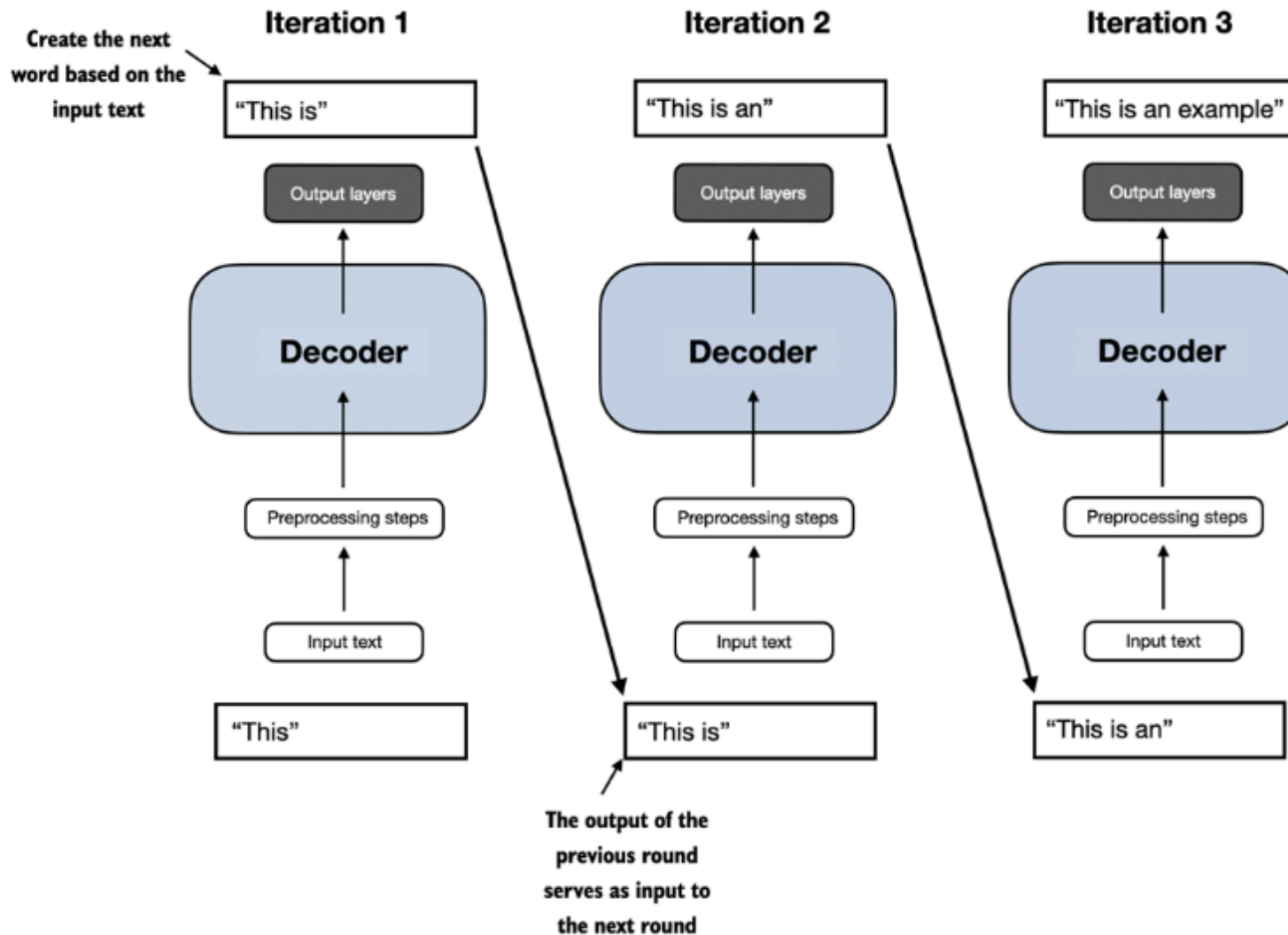
source: news reports, [LifeArchitect.ai](https://lifeai.com/)
* = parameters undisclosed // see [the data](#)

MADE WITH [VIZsweat](#)

What about ChatGPT ?

Only decoder type LLM indeed, but the output improvement relies on the concept of RLHF (Reinforcement Learning with Human Feedback)

ChatGPT structure



Some LLM news - 2023!

- Multimodality already for some!
- As apps (ChatGPT android)
- Specific: ChatGPT can see, hear and speak ("ChatGPT can now see, hear, and speak.
- [Custom GPTs](#) are released already! More recently, they released [GPT store](#)
- Upcoming: "[Gemini](#) is built from the ground up for multimodality — reasoning seamlessly across text, images, video, audio, and code."

Some LLM news - 2024!

- Different versions of GPT (including 4o, mini, o1-preview for advanced reasoning), lastly with GPT-4o with canvas
- Voice option is working now for GPT and more recently introduced ChatGPTSearch
- Claude 3.5 Sonnet, and Claude 3.5 Haiku - can direct Claude to use computers the way people do
- Meta's Next Llama AI Models Are Training on a GPU Cluster 'Bigger Than Anything' Else ?
- Increase on the multi-agent frameworks for different applications!

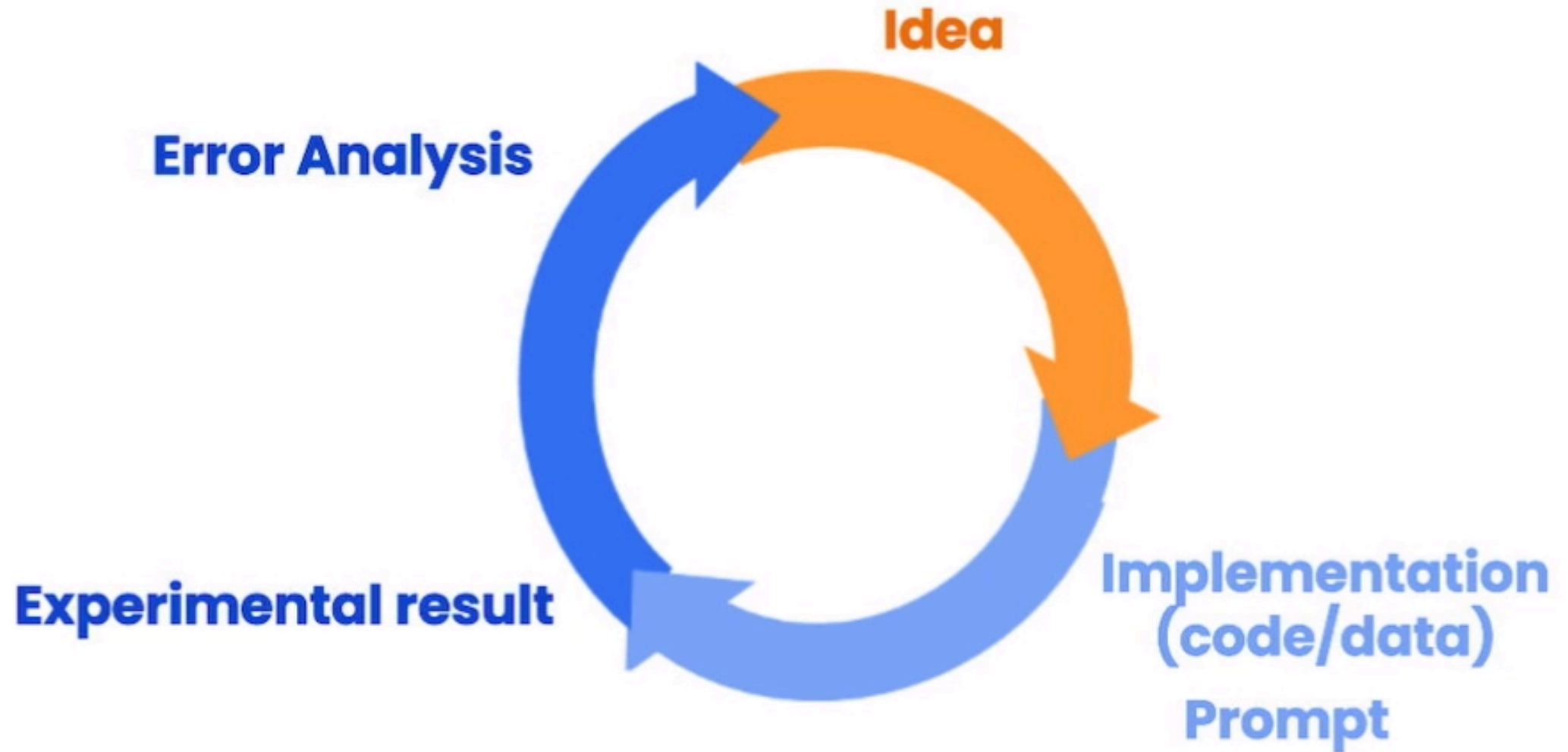
What about available tools that we can play with ?

Which tool we can use?

- OpenAI: **GPT-3.5** or recent version **GPT4** if you have paid membership
- Google: **Bard** now, **Gemini** later. For [Bard](#)
- Anthropic: **Claude-2** with its [website](#)
- Meta: **Llama-2** (70B) via a specific [website](#)
- Mistral: Check out their [platform as open-source alternative](#)
- [Microsoft Copilot](#) is available for university! announced as *Protected Mode for users from educational institutions*
- *University ELM platform*: Edinburgh (access to) Language Models, via [ELM](#)

Prompt Engineering

Iterative Prompt Development



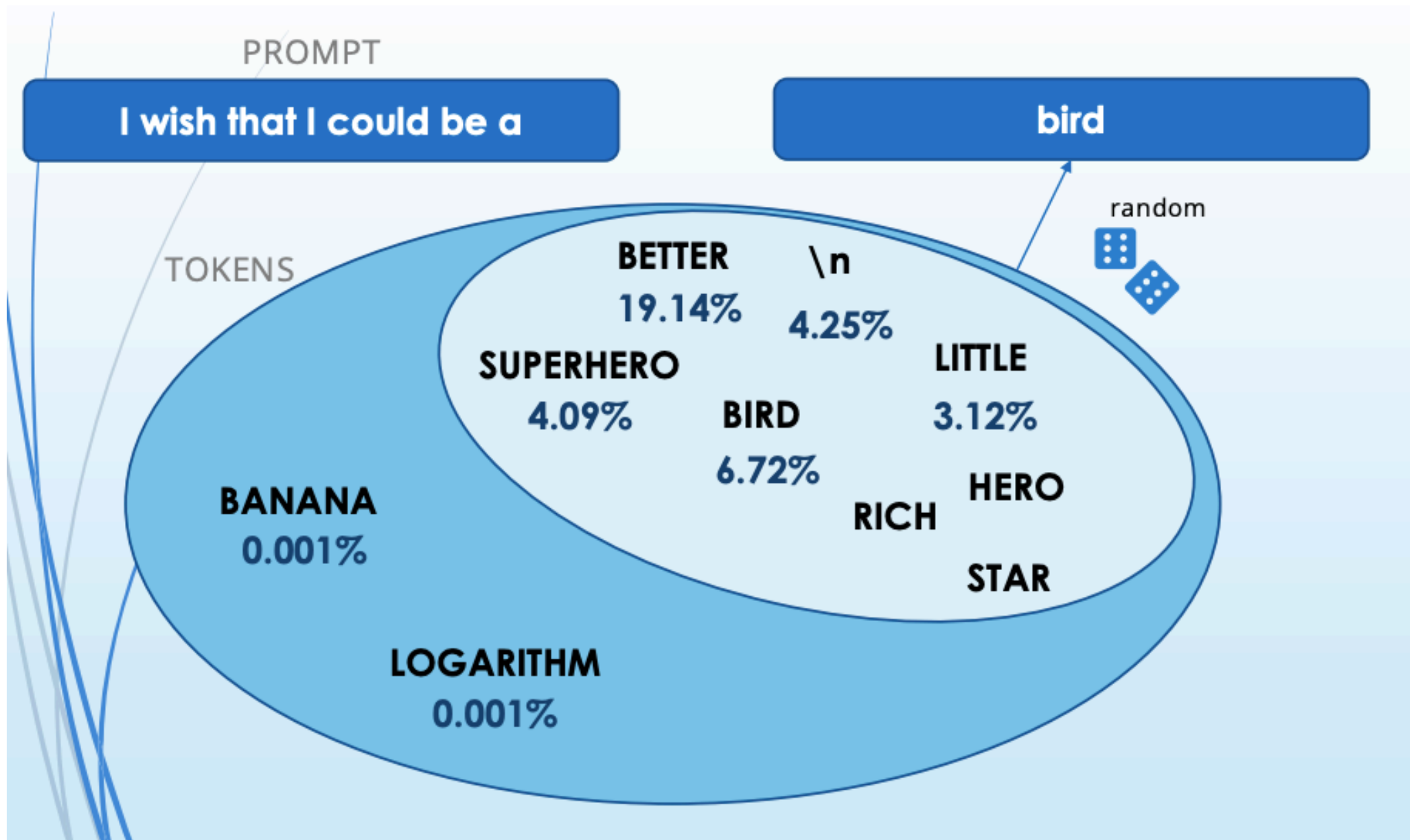
About LLM settings

- When designing and writing prompts, we typically interact with the LLM via an API or more friendly interfaces such as ChatGPT etc.
- It is crucial to know configured model parameters and how they work in general!

What about these main parameters ?

LLM Model Parameters

- *Max Length*
 - Controls output token count. Helps avoid overly long or irrelevant responses, managing cost.
- *Stop Sequences*
 - Strings that terminate token generation. Used to limit response length and structure.
- *Frequency Penalty*: Penalizes repeated tokens based on frequency. Reduces word repetition in responses.
- *Presence Penalty*: Penalizes all repeated tokens equally. Prevents phrase repetition. Adjust for creativity or focus.



 Note

The general recommendation is to alter **temperature** or **Top P** but **not both**.

 Note

Similar to temperature and Top P, the general recommendation is to alter the **frequency** or **presence** penalty but **not both**

 Warning

Keep in mind that your results may vary depending on the which LLM or which version of LLM you use.

Some **further details from OpenAI**

Basic Template

- **Prompt:** What we want from LLM
- **Output:** What we will get from the LLM

Note

A prompt can contain information like the instruction or question you are passing to the model and include other details such as context, inputs, or examples

Tip

It is possible to use such elements when they are necessary to guide the LLM more effectively, for the purpose of output improvement!

Elements of a Prompt

- **Instruction:** a specific task or instruction you want the model to perform
- **Context:** external information or additional context that can steer the model to better responses
- **Input Data:** the input or question that we are interested to find a response for
- **Output Indicator** the type or format of the output.

Warning

No need all the four elements for a prompt and the format depends on the task.

Some Examples

- Text Summarization
- Information Extraction
- Question Answering
- Text Classification or Sentiment Analysis
- Code Generation

ELM Use-cases

Examples demonstrating the application of ELM across different themes;

- Code Writing
- Writing a Literature Review
- Summarising a material
- Searching for academic information
- Learning a subject
- Developing Teaching Plans
- Creating Assessments
- Generating Research Ideas
- Checking response to tutorial questions
- Create Templates

OpenAI suggestions

- Write clear instructions
- Provide reference text, if we need
- Split complex tasks into simpler sub-tasks
- Give the model time to “think”
- Use external tools or knowledge, if we need
- Test changes systematically

Note

For each strategy we can see various given tactics below

Write clear instructions

- Include details in your query to get more relevant answers
- This is like feeding LLM with important details or context
- Ask the model to adopt a persona
 - You act as a workshop tutor!
- Use delimiters to clearly indicate distinct parts of the input
 - specify the related content explicitly, ie.
insert first article here
- Specify the steps required to complete a task
 - Splitting the task into smaller parts
- Provide examples
- Specify the desired length of the output

Provide reference text

- Instruct the model to answer using a reference text
 - Adding extra information
- Instruct the model to answer with citations from a reference text
 - Adding external source

Split complex tasks into simpler subtasks

- Use intent classification to identify the most relevant instructions for a user query
- For dialogue applications that require very long conversations, summarize or filter previous dialogue
- Summarize long documents piecewise and construct a full summary recursively

Give models time to “think”

- Instruct the model to work out its own solution before rushing to a conclusion
- Use inner monologue or a sequence of queries to hide the model’s reasoning process
- Ask the model if it missed anything on previous passes

Use external tools

- Use embeddings-based search to implement efficient knowledge retrieval
- More technical, such as RAG method
- Use code execution to perform more accurate calculations or call external APIs
- Such as guiding LLM to do something with another tool!
- Give the model access to specific functions

Testing changes

Evaluation procedures (or “evals”) are useful for optimizing system designs

- Evaluate model outputs with reference to gold-standard answers

Note

More advanced prompting engineering techniques that allow us to achieve more complex tasks and improve reliability and performance of LLMs

Risks & Misuses

- Adversarial Prompting
- Factuality
- Biases

Note

Remember that LLM outputs may include hallucination in general!

Advanced Methods

These are some techniques from the open book of [Prompt Engineering Guide](#). Including but not limited to,

- Zero-Shot Prompting
- **Few-Shot Prompting**
- **Chain-of-Thought Prompting**
- Generated Knowledge Prompting
- PAL (Program-Aided Language Models)
- **Retrieval Augmented Generation (RAG)**

Few-shot learning

- Generally, LLMs are capable of performing some tasks “zero-shot.” Just asking what we want simply
- When zero-shot doesn’t work, it’s recommended to provide demonstrations or examples in the prompt which leads to few-shot prompting
- This technique enables in-context learning where we provide demonstrations in the prompt to improve the model performance.
- For more difficult tasks, we can experiment with increasing the demonstrations (e.g., **3-shot**, 5-shot, 10-shot, etc.)

Warning

It is better than zero-shot to some extent but may fail especially when dealing with more complex reasoning tasks!

Chain of Thought (CoT)

- This method has been popularized to address more complex arithmetic, commonsense, and symbolic reasoning tasks.
- Chain-of-thought (CoT) prompting enables complex reasoning capabilities through intermediate reasoning steps ([Wei et al. \(2022\)](#))
- It is possible to combine it with few-shot prompting to get better results on more complex tasks
- You can see different types such as Zero- or Few-shot CoT Prompting or Automatic Chain-of-Thought (Auto-CoT)

CoT Example

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

More technical: RAGs

- For more complex and knowledge-intensive tasks, it's possible to build a language model-based system that accesses external knowledge sources
- This is a common and one of the cheapest option to reduce the problem of “hallucination” in general
- RAG combines an information retrieval component with a text generator model.
- RAG allows language models to bypass retraining, enabling access to the latest information for generating reliable outputs via retrieval-based generation

Workflow of RAG

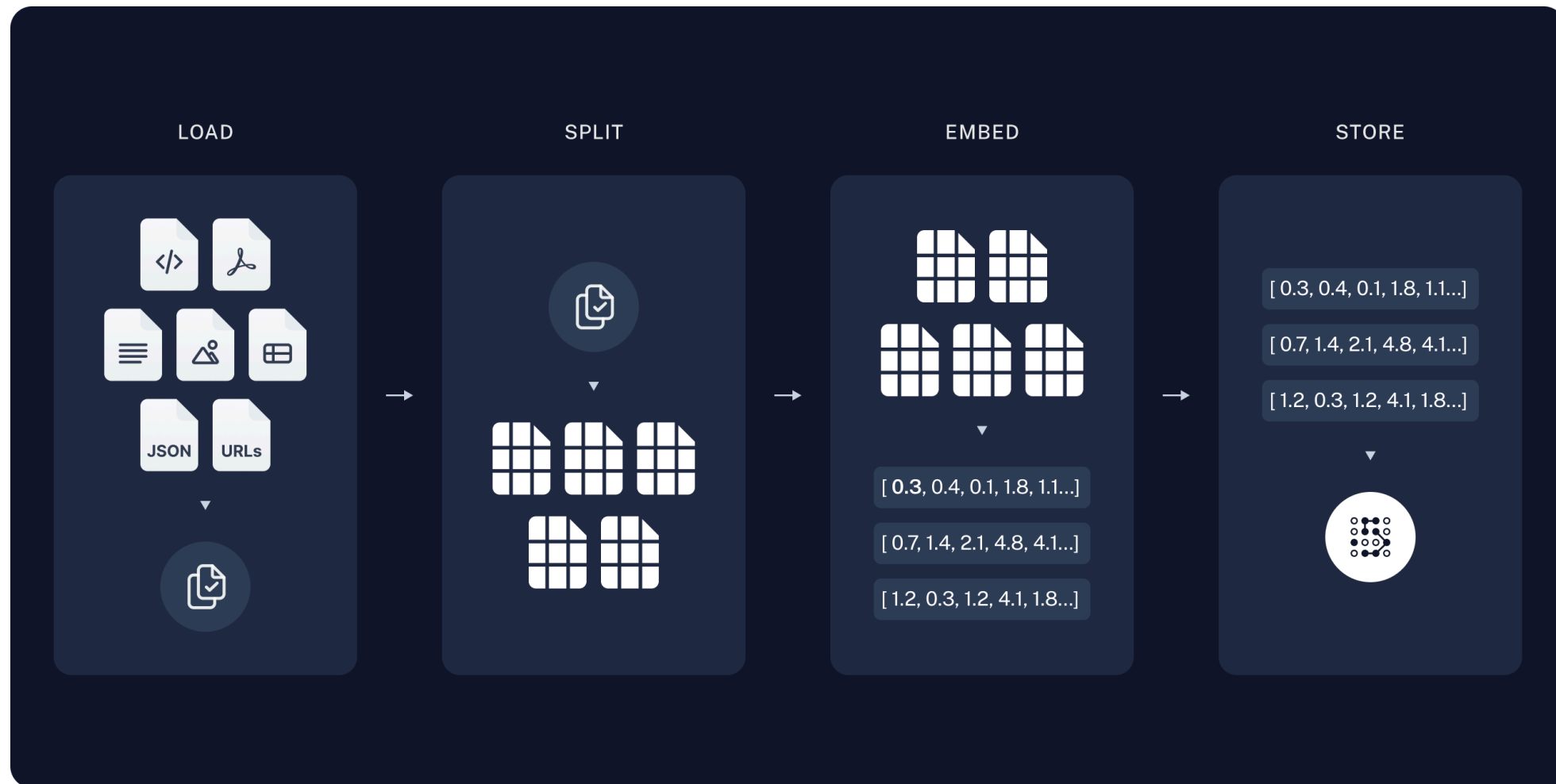


Image Credit: [Langchain documentation](#)

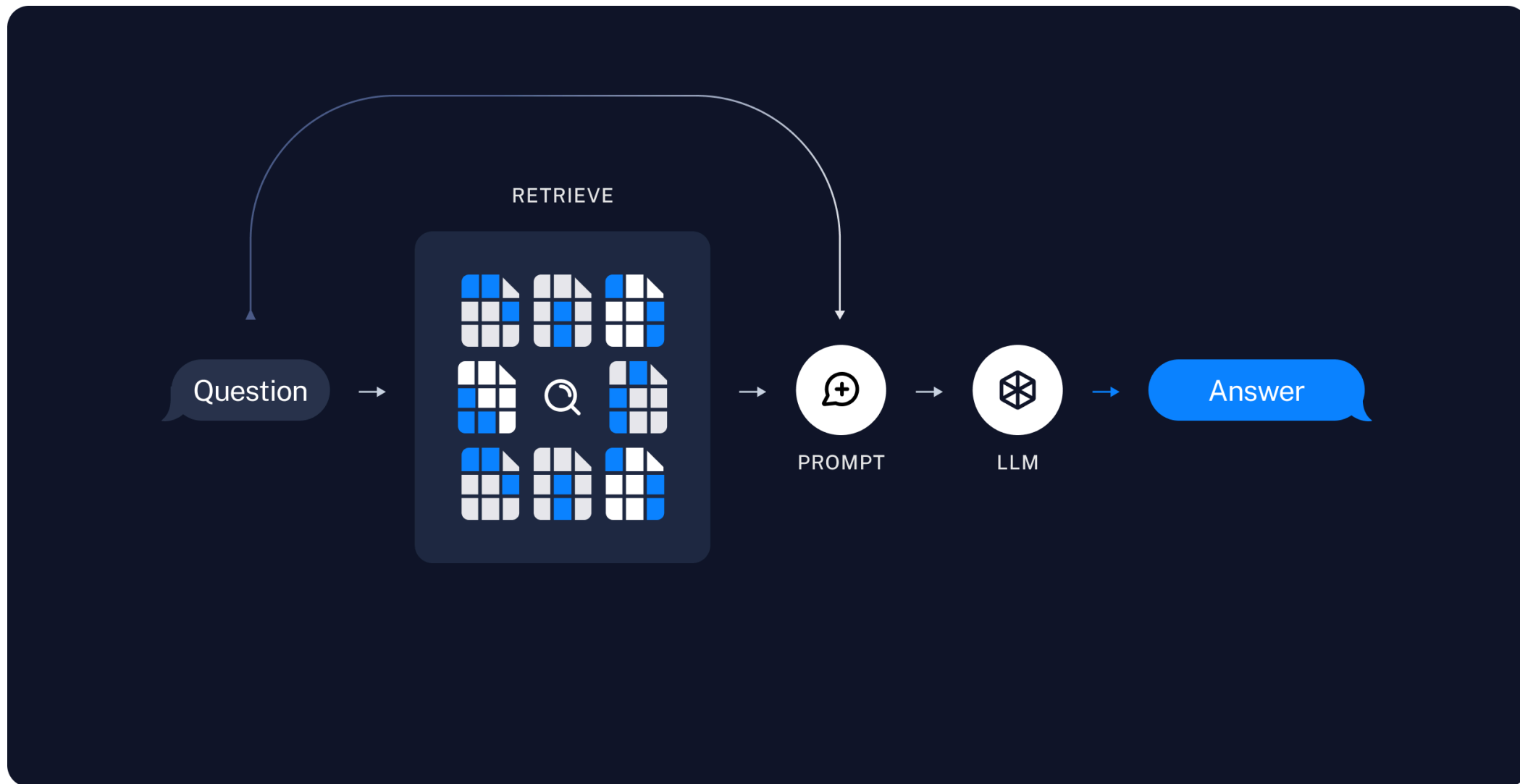


Image Credit: [Langchain documentation](#)

There are some code-based examples (if you are interested)

- Short notes and examples in Python
- Short demo of Embedchain for a webpage
- Short demo of Langchain for a pdf file

About GPTs via OpenAI

Note

Brief Introduction to ChatGPT Advanced Data Analytics and about GPTs!

“Discover and create custom versions of ChatGPT that combine instructions, extra knowledge, and any combination of skills.”

Other LLM related tools can be;

- Scholar ai -
- Elicit - Analyze research papers
- Perplexity AI - Conversational search engine
- ...

Other Resources on Prompting

- promptingguide.ai: A prompt engineering guide that demonstrates many techniques. Especially the [hub page](#) can be good to explore
- [OpenAI Cookbook](#)
- [OpenAI Prompt Examples](#)
- [Prompting libraries & tools](#)

- [Google Prompt Engineering page](#)
- [How to Use DALL-E 3: Tips, Examples, and Features](#)
- [Brex's Prompt Engineering Guide](#) Brex's introduction to language models and prompt engineering.

Time to Reflect

Please share your feedback to improve this training

